

The Algorithm Journey

- **Decision:** Starting with Random Forest as a baseline.
- **Reason:** It is highly resistant to overfitting and serves as a great "sanity check" to ensure the pipeline is working, but it usually lacks the predictive power needed for enterprise applications.
- **Decision:** Testing XGBoost with One-Hot Encoding.
- **Reason:** XGBoost is an industry standard for tabular data. However, using One-Hot Encoding on the `variant` column (which had over 2,000 unique values) created a "sparse matrix"—thousands of new columns filled mostly with zeros. This diluted the importance of actual features like `age` and caused XGBoost to project an illogical, crashing depreciation curve.
- **Decision:** Selecting CatBoost as the final production engine.
- **Reason:** CatBoost is engineered specifically to handle high-cardinality categorical data natively. By bypassing One-Hot Encoding, CatBoost grouped the text strings mathematically on the fly. This resulted in an R^2 accuracy above 0.91 and produced a beautifully smooth, realistic buy-back depreciation curve.

Model Iteration	Strategy	R-Squared Score	Mean Absolute Error	Error Margin	Functional Verdict
Random Forest	Baseline preprocessor with One-Hot Encoding	0.8136	₹83,671	14.80%	High variance; struggled with high-cardinality categorical variants.
XGBoost	Optuna Hyperparameter optimization	0.9184	₹65,731	12.00%	Top statistical score, but generated an illogical, crashing future depreciation curve due to sparse matrix creation.

CatBoost	Native Categorical Feature Processing + Optuna	0.9140	₹66,430	12.13%	Selected Engine. Handled textual features natively, resulting in smooth, highly realistic real-world buy-back curves.
-----------------	--	--------	---------	--------	--

Hyperparameter Tuning

- **Decision:** Using Optuna for Bayesian hyperparameter optimization.
- **Reason:** Hardcoding parameters (like guessing `depth=8`) limits a model's potential, and traditional GridSearch is too slow and rigid. Optuna runs automated, intelligent experiments, learning from previous trials to hunt down the exact mathematical ceiling of your specific dataset. This single decision dropped your error margin down to ~12%.

The Buy-Back Forecasting Logic

- **Decision:** Using cross-sectional aging simulation rather than time-series forecasting.
- **Reason:** To forecast future prices, you typically need historical tracking data (e.g., watching a specific car's price drop month over month). Your dataset was cross-sectional—a snapshot of prices *today*. To bypass this limitation, we engineered a function to artificially increase the `age` and `mileage` variables of the user's input. We are essentially asking the model: *"Based on today's market, what is a car with these exact specs worth if it were 3 years older and had 36,000 more kilometers?"*

Because the dataset is a cross-sectional snapshot of current prices (lacking longitudinal time-series data), future valuation is calculated using a **cross-sectional aging simulation**.

To predict a vehicle's value at future time horizon **t**(in years):

- Isolate the current feature vector (Make, Model, Variant, Mileage, Age).
- Artificially increment the age variable: $\text{Age}_{\text{future}} = \text{Age}_{\text{current}} + t$
- Simulate mileage accumulation using a constant: $\text{Mileage}_{\text{future}} = \text{Mileage}_{\text{current}} + (12000 * t)$
- Force the modified specifications through the trained CatBoost pipeline.
- Apply the inverse log transformation to extract the actual forecasted currency value.

Architecture & Deployment Strategy

- **Decision:** Abandoning the ONNX export format.
- **Reason:** While ONNX is great for running models directly in Node.js, CatBoost's C++ source code currently has a hard limitation: it cannot export categorical text trees to ONNX. To force an ONNX export, we would have had to convert all text to integers, breaking CatBoost's native text-handling superiority and pushing mapping complexities onto your frontend.
- **Decision:** Deploying via a Python FastAPI Microservice.
- **Reason:** This is the enterprise standard. By keeping the model in its native `.cbm` format and wrapping it in a lightweight Python API, the model retains its maximum speed and accuracy. It securely decouples your heavy machine learning math from your Node.js/Next.js web application, allowing your frontend to remain incredibly fast and responsive.

CatBoost Buyback Prediction API

Start the Local API Server

First, you need to run the Python server that contains your exported CatBoost model.

1. Install the Required Server Libraries

Open your local terminal and run the following command:

```
pip install fastapi uvicorn catboost pandas pydantic numpy
```

2. Create the Server File

Create a file named `main.py` in the exact same folder where you downloaded your `catboost_pricing_model.cbm` file.

The folder structure should look like this:

```
project-folder/  
├──  
├── main.py  
└── catboost_pricing_model.cbm
```

Paste the code from `main.py`:

3. Run the Server

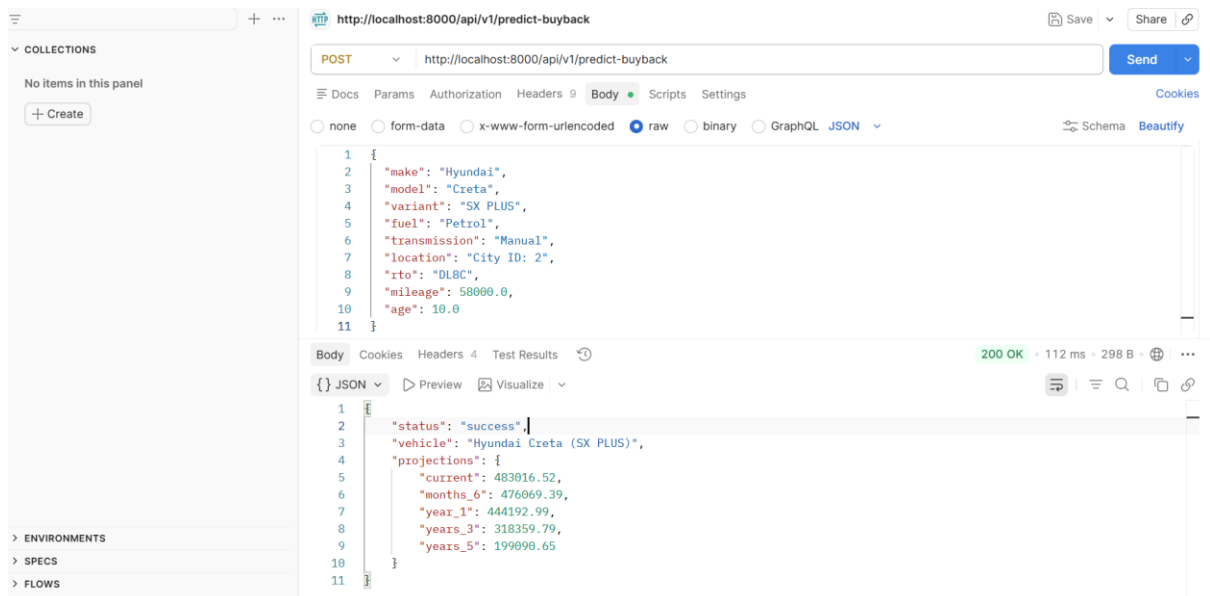
In the same terminal, execute:

```
python main.py
```

If everything is configured correctly, you should see a message similar to:

Uvicorn running on http://0.0.0.0:8000

This means that your local API server is running successfully on port 8000.



The API automatically increases the vehicle's **age** and **mileage** for future projections using the configured yearly mileage assumption of **12,000 km/year**.